

RED TEAM EXERCISE — ROUND 2

Corner Voice — Marketing Site Backend & Frontend

Addendum to the original Red Team Exercise Report — closing the gaps that report explicitly named as untested

Prepared by: Milton Evensizer

Engagement dates: Late August 2026

Classification: Confidential — Portfolio / Educational Use

1. Authorization & Engagement Statement

Self-authorized and self-conducted, against the author's own codebase, on the same basis stated in the original audit report and the Round 1 red-team report. Not an independent third-party engagement.

2. Executive Summary

The original red-team report (Round 1) explicitly scoped itself to the AI service logic (`services/ai.js`) and explicitly named several areas as untested: the marketing site's own backend wrapper code (contact form, demo endpoint, rate limiting, CORS), the frontend's own rendering logic, dependency vulnerabilities, and whether the original audit's F-01 and F-03 fixes actually hold under real re-attack rather than just review. This round closes those specific gaps.

One dependency scan, one re-attack of a prior fix, and six new adversarial test categories were run. One category (dependency scan) came back clean. One prior fix (F-03/IDOR) was re-attacked and confirmed holding. One prior fix (F-01/RLS) could not be adversarially re-tested — see Section 5. Four new categories (CORS, contact form injection, demo endpoint input handling, npm audit) held without any code changes. Two real issues were found and fixed: a DOM-based XSS in the frontend, and a missing reverse-proxy trust configuration that would have broken rate limiting in production.

Methodology Note: A Self-Correction Worth Documenting Directly

The DOM-XSS finding in this report (RT2-01) involved two fix attempts, not one, and that is documented in full rather than only showing the final, working state. The first fix applied the wrong escaping function for the HTML context involved; re-running the identical attack against that first fix showed it was still exploitable. A second, corrected fix was then applied and verified two separate ways: direct DOM attribute inspection, and an actual simulated mouse-hover interaction producing zero script execution. Separately, the verification process itself produced one false-positive result (a naive string-match check flagged the corrected fix as still vulnerable, when in fact it had already worked) — caught by switching to DOM-level verification instead of raw-HTML string matching. Both corrections are included here because a report that only shows the final clean state would misrepresent how this finding was actually resolved.

3. Scope

Test type: adversarial, whitebox, against the marketing site's backend (`marketing_backend/`) and frontend (`cv_website/`) codebases, plus a re-attack of two findings from the original audit report.

- Re-attack: F-03 (IDOR on job-polling endpoint) from the original audit — never previously re-tested after its fix
- Re-attack: F-01 (RLS bypass) from the original audit — attempted; see Section 5 for why this could not be completed
- CORS configuration (`src/index.js`)
- Contact form input handling (`src/routes/contact.js`)
- Demo endpoint input handling (`src/routes/demo.js`)
- Rate limiter / reverse-proxy trust configuration (`src/middleware/rateLimiter.js`, `src/index.js`)

- Frontend result rendering (cv_website/assets/tryit.js)
- Dependency vulnerability scan (npm audit) — not previously run at any point in this project

4. Findings Summary

ID	Finding	Area	CVSS	Risk Rating	Status
RT2-01	DOM-based XSS via unescaped class attribute	Frontend	3.1	High	FIXED
RT2-02	Missing reverse-proxy trust configuration	Backend Reliability	N/A††	Medium	FIXED

† RT2-01's CVSS Base Score (3.1) is well below its Overall Risk Rating (High) — see the finding's own Severity Rationale below for why, following the same divergence-transparency practice established in the original audit report.

†† RT2-02 is a control-effectiveness / reliability defect, not a classically exploitable vulnerability with a meaningful CVSS profile — an attacker does not "exploit" a missing trust-proxy setting to gain anything; the defect degrades an existing protection's effectiveness in production. Scored qualitatively only, consistent with how F-08 was handled in the original audit report.

RT2-01 DOM-Based XSS via Unescaped Class Attribute**HIGH****CVSS 3.1****Location:** cv_website/assets/tryit.js — chip()**CVSS v3.1 Vector:** CVSS:3.1/AV:N/AC:H/PR:N/UI:R/S:U/C:N/I:L/A:N**Description**

The chip() function, used to render sentiment/urgency labels returned by the demo API, interpolated its cls parameter directly into an HTML class attribute without escaping. The text parameter (the visible label content) was correctly escaped via escapeHtml(); the cls parameter, used to build the CSS class name from the same backend-provided values, was not.

Attack & Result — First Confirmation

A live browser test (not a static code review) simulated a backend response containing a crafted sentiment value designed to break out of the class attribute and inject a live onmouseover handler. Direct inspection of the resulting DOM confirmed a genuine, executable onmouseover attribute was present on the rendered element.

```
sentiment: 'positive" onmouseover="alert(\'XSS\')" x=""
```

Root Cause

No output-context escaping existed for the class-attribute value at all — the field was assumed safe because the backend's analyzeReview() constrains sentiment/urgency to a fixed enum before returning them. That backend guarantee is real and currently holds, but the frontend had zero independent defense if it were ever bypassed by a future code change, a different code path, or a compromised dependency.

Fix Attempt #1 — Applied, Then Found Still Vulnerable on Re-Test

The first fix wrapped cls in the existing escapeHtml() function, the same one already used for the text content. Re-running the identical attack against this fix showed it was still exploitable. Root cause of the failed fix: escapeHtml() is implemented via a.textContent → innerHTML round-trip, which only escapes characters with special meaning in HTML TEXT CONTENT (<, >, &). It does not escape double-quote characters, because quotes have no special meaning in text content — but cls is inserted into an HTML ATTRIBUTE value, where the quote character is exactly what allows a value to break out. The same escaping function is correct everywhere else it is used in this file (all other uses insert content as text between tags, not into attribute values) — it was simply the wrong tool for this specific context.

Fix Attempt #2 — Corrected, Verified

A dedicated escapeAttr() function was added, explicitly escaping &, <, >, ", and ' by string replacement rather than relying on the DOM's text-content serialization behavior. Applied specifically to the cls parameter.

Re-Test Result

Verified two ways against the identical attack payload: (1) direct DOM inspection confirmed hasAttribute('onmouseover') returns false on the rendered element; (2) an actual simulated mouse-hover

interaction over the live element produced zero script execution. A naive string-match check on the raw HTML initially and incorrectly suggested the fix had failed — the crafted text was still visible as inert page text, which the string-match check could not distinguish from a live attribute. Switching to DOM-level verification resolved the false positive and confirmed the fix is genuinely effective.

Severity Rationale (CVSS vs. Overall Risk Rating)

CVSS Base Score (3.1) is Low primarily because Attack Complexity is scored High (this specific path currently requires the backend's enum validation to already be bypassed) and Confidentiality impact is None (this public demo page has no session or authentication data to steal). The Overall Risk Rating is held at High because this is a defense-in-depth failure on a page whose entire purpose is to demonstrate the platform's security posture to a prospective customer — the reputational and trust cost of this specific class of bug existing on this specific page is disproportionate to its narrow technical exploitability today.

RT2-02 Missing Reverse-Proxy Trust Configuration	MEDIUM	No CVSS
--	--------	---------

Location: marketing_backend/src/index.js

Description

Express's trust proxy setting was never configured. Without it, req.ip — the value express-rate-limit uses as its per-visitor key — returns the connecting socket address rather than the real client IP whenever the app runs behind a reverse proxy, which it always will in production (Render).

Attack & Result

Rather than reason about this abstractly, spoofed X-Forwarded-For headers claiming two different attacker-chosen IPs were sent against the unmodified code. Neither had any effect on which rate-limit bucket the requests were counted against — confirming req.ip was ignoring the header entirely, exactly as the missing configuration would predict.

Real-World Impact — Not an Exploit, a Reliability Failure

This is not a vulnerability an attacker exploits to gain something. It is a defect that would silently degrade an already-working protection once deployed: every real visitor to the live site, regardless of their actual IP address, would resolve to Render's single proxy address, and the intended "5 requests per hour per visitor" limit on the public demo would become "5 requests per hour, shared across all site traffic combined" — the demo would appear broken for every legitimate visitor after the first 5 uses each hour, site-wide.

Fix Applied

- Added app.set('trust proxy', 1), matching Render's single-reverse-proxy architecture — trusting exactly one hop, not an arbitrary number

Re-Test Result

Re-run with simulated X-Forwarded-For values representing two different real visitors: each now receives their own independent 5-request bucket, and a 6th request from the same simulated visitor correctly returns 429 while a different visitor's requests are unaffected.

Limitation of This Verification

A local test cannot fully replicate the actual production trust boundary. The property that matters in production — that only Render's real proxy, not an external attacker connecting directly, can set the trusted X-Forwarded-For value — only exists once genuinely deployed behind Render's infrastructure. What was confirmed here is that the mechanism itself now works correctly (distinct visitors get distinct buckets); full adversarial confirmation of the production trust boundary is a live-deployment test, not a local one — the same category of limitation documented for F-01/RLS in Section 5.

5. Re-Attacked Prior Fix, Confirmed-Secure Controls, and What Could Not Be Tested

5.1 — F-03 (IDOR), Re-Attacked and Confirmed Holding

The original audit's fix for F-03 was never adversarially re-tested until this round. A direct attack was run: Job created and owned by a simulated User A; a simulated User B then attempted to retrieve that job by ID. Result: blocked with a 404 indistinguishable from a genuinely nonexistent job (not a 403 that would confirm the job's existence to an attacker) — confirmed via the actual ownership-check code path, not just a review of the fix.

5.2 — F-01 (RLS), Attempted, Could Not Be Adversarially Confirmed

The code fix (using the Supabase anon key rather than the service-role key in `supabaseForUser()`) was confirmed present via direct code inspection. Whether RLS itself actually enforces correctly cannot be adversarially tested without a live Postgres/Supabase instance running the real schema and policies — no such instance currently exists for this project. This is stated as an explicit, unresolved gap, not implied to be covered by the static check alone.

5.3 — CORS, 5 Bypass Techniques Attempted, All Blocked

- Substring/prefix trick (evil domain containing the allowed origin) — blocked
- Suffix trick (allowed origin embedded in a longer malicious value) — blocked
- null origin (sandboxed iframe / file:// simulation) — blocked
- Case-sensitivity variation — blocked
- (Requests with no Origin header at all succeed by design — this is not a CORS bypass, since CORS is a browser-enforced mechanism that never restricted non-browser clients from reaching a public, unauthenticated endpoint in the first place. The actual protections for that scenario are the rate limiter and Turnstile, not CORS.)

5.4 — Contact Form Input Handling

- CRLF/email-header-injection attempt — blocked by email format validation
- Oversized message field — blocked by length validation
- HTML/script injection in the name field — accepted as valid input (correct; a name field cannot reasonably reject HTML-shaped text), and confirmed neutralized in the actual constructed email HTML via `escapeHtml()`, tested directly against the real email-building code, not assumed
- Type confusion (message sent as an object instead of a string) — blocked
- `__proto__` key injection attempt — accepted harmlessly; `JSON.parse` (used internally by Express's body parser) does not trigger prototype-setter behavior the way object-literal syntax does, so this did not constitute prototype pollution

5.5 — Demo Endpoint Input Handling

- Rating as a numeric string ("5") — accepted via intentional `Number()` coercion, a reasonable design choice, not a defect
- Rating as a float, out-of-range, and negative — all correctly rejected

- Oversized review text — correctly rejected
- reviewText sent as an array instead of a string — correctly rejected, and confirmed the type check is ordered before any string-method call that would otherwise throw an unhandled exception on non-string input
- Oversized business-name field containing an XSS-shaped payload — correctly rejected by length validation before the content itself was ever evaluated

5.6 — Dependency Scan

npm audit against marketing_backend's current dependencies: 0 known vulnerabilities at any severity level.

6. Conclusion & Effect on Public Security Claims

This round closes every gap the original Round 1 report explicitly named as out of scope, with one exception that remains structurally untestable without live infrastructure (F-01/RLS, and the full production trust-boundary property of RT2-02) rather than one that was simply skipped.

Two real issues were found and fixed. One (RT2-01) required two attempts to fix correctly, and that process — including a false-positive verification result caught and corrected — is documented in full rather than only presenting the clean final state, consistent with the self-correction already on record in the Round 1 report.

Effect on the Corner Voice Security page's existing claims: the specific figures already published ("8 findings identified & fixed," "8 adversarial attacks tested, 8 blocked") remain accurate as statements about the audit and Round 1 exercise specifically. They do not yet reflect this round's findings. Whether and how to update that page's language to reference this round is a separate decision from this report's findings themselves.